

Secretary HQ — Product Roadmap & Task Handoff

This document is a work-order backlog, not a wish list. Every task below is written so that an AI engineer (Claude) or a human can pick it up cold, execute it, and prove it is done with a command, query, or named test — no subjective judgment.

Roles:

- **Architect / PM** (the author of this doc): defines tasks, acceptance criteria, dependencies, and order.
- **Implementer** (Claude, or a human dev): claims a task, does the work, and satisfies its Definition of Done.

Golden rule for implementers: A task is DONE only when its **Acceptance Test** passes. If you cannot run the acceptance test, the task is NOT done — say so.

0. COMPLETE CONTEXT (read this before claiming any task)

0.1 What the product is

Secretary HQ is an AI phone receptionist for local service businesses (auto shops, salons, trades, fitness, food & beverage). It answers inbound calls 24/7, books appointments atomically, answers policy questions from a knowledge base, takes messages, and logs every call (transcript + AI summary) to an owner dashboard.

Brand promise: “Sounds Real. Books Smart. Never Misses a Call.” — live in under 10 minutes.

0.2 Architecture (the map you need to navigate the repo)

```
Inbound Call
→ Telnyx (carrier + SIP trunk)
→ LiveKit Cloud (SIP ingress + agent orchestration)
→ LiveKit Agent worker (Node) [agent/]
  - Deepgram Nova-3 (STT)
  - OpenAI GPT-4.1-mini (voice LLM)
  - Deepgram Aura (TTS, streaming)
  - Call flow = QUESTION TREES (agent/src/checklist/)
→ Fastify backend (29 route modules) [src/]
  - JWT auth, Zod validation, RLS via withTenantClient()
→ PostgreSQL + pgvector (Row Level Security) [supabase/]
→ Next.js 16 dashboard [dashboard/]
```

Layer	Path	Stack
Backend	src/	Fastify 5, TypeScript, Zod, JWT
Agent (voice)	agent/	LiveKit Agents (Node), Deepgram, OpenAI
Dashboard	dashboard/	Next.js 16 (App Router), React 19, Tailwind
Database	supabase/	PostgreSQL + pgvector, 190 migrations, RLS
Shared	shared/	Cross-runtime code (derivation, scheduling, embeddings)

0.3 How the call flow works (CRITICAL — most tasks touch this)

The live call path is **question trees**, NOT a free-form LLM. See `agent/src/checklist/` :

- **Trees** (`trees.ts` , `verticalIntakeTrees.ts`): `QuestionTreeDef` — ordered nodes the agent walks.
- **Blocks** (`blockLibrary.ts`): `ConversationBlockDef` — reusable units that reference trees.
- **Presets** (`presets.ts`): `VerticalPresetDef` — per-business-type bundles of blocks.
- **Derivation** (`shared/checklistPresetDerivation.ts`): maps `business_type slug` → `preset_id` → runtime config.
- **Passive slot-filling**: nodes with `listen:true` capture volunteered info but never interrogate the caller and never gate the goodbye.
- Each tree must have exactly **one active top-level lead node**; the rest are `listen:true` .

0.4 How to build, test, and run (implementers MUST use these exact commands)

```
# Full local setup (installs deps, starts Docker Postgres, migrates, seeds, tests)
npm run bootstrap

# Run the three test suites (all must be green before any merge)
npm test                # backend / root (Vitest)
cd agent && npm test      # agent (Vitest)
cd dashboard && npx vitest run # dashboard (Vitest)
cd dashboard && npx playwright test # E2E (40 spec files)

# Typecheck
npm run typecheck      # (and in agent/ and dashboard/)

# Start dev servers
npm start              # dashboard https://localhost:4000, backend https://localhost:4001

# CLAUDE.md drift check (CI gate – migration count must match files on disk)
npm run verify:claude-md
```

0.5 How shipping works (deploy model)

- All 3 Railway services (`secretary-hq` , `secretary-hq-agent` , `dashboard`) deploy from `main` .

- **Shipping = merging a PR to `main`** . A push to a feature branch deploys nothing.
- `main` is branch-protected: **all 4 CI jobs must be green** to merge.
- **A red `main` CI run makes Railway mark that commit's deploy SKIPPED, and SKIPPED is terminal** — turning CI green later does NOT retry. So flaky CI is an availability bug, not a nuisance.
- Apply production DB migrations per the order rule in each task (default: migration before merge, unless the task says otherwise).

0.6 HARD CONSTRAINTS (never violate — these are permanent policy)

1. **No HIPAA / PHI verticals.** `dentist` , `chiropractor` , `vet-clinic` were removed by migration `20260321000000_remove_hipaa_templates.sql` . Do not re-add health verticals or a “HIPAA tier” without explicit owner sign-off. There are **30 active non-HIPAA verticals**.
2. **Voice = question trees.** Do not replace the tree flow with a free-form LLM agent.
3. **SMS is OFF by design** until 10DLC registration completes. `ENABLE_SMS` defaults to false in the agent config schema. “Fixing SMS” means completing 10DLC + flipping the gate, not patching a bug.
4. **No live human transfer on the question-tree path today.** Escalation takes a message and flags urgency. SIP REFER plumbing exists but is not wired into the tree flow. Do not claim transfer works until a task explicitly builds and tests it.
5. **Product voice** in all user-facing copy: “Sounds Real. Books Smart. Never Misses a Call.” / “Live in under 10 minutes.”
6. **Every PK follows `<table_singular>_id`** (see `docs/CODING_STANDARDS.md`).
7. **Every test covers happy + sad paths** with 5W diagnostic context (Who/What/When/Where/Why) in sad-path assertions.

0.7 Status legend (how to read + set task status)

Each task carries a `STATUS:` line. Allowed values, and **how status is objectively determined**:

Status	Meaning	How it's proven
<code>NOT_STARTED</code>	No work begun	No branch, no commits
<code>IN_PROGRESS</code>	Being worked	A branch exists; acceptance test not yet passing
<code>BLOCKED</code>	Waiting on a dependency or external gate	The blocker is named in the task
<code>DONE</code>	Complete + verified	The Acceptance Test passes and the change is merged to <code>main</code>

An implementer may only mark a task DONE by pasting the passing output of its Acceptance Test into the PR.

0.8 Task anatomy (every task below uses this exact template)

```

### T-XXX: <title>
STATUS: <status>
OWNER: <Claude-able | Human-only | Mixed>   ← Human-only = needs external account/
phone/bank; Claude cannot do it
PRIORITY: <CRITICAL | HIGH | MEDIUM | LOW>
EFFORT: <hours estimate>
DEPENDS_ON: <task IDs or None>
CONTEXT: why this exists + current state
FILES: exact paths to touch
STEPS: ordered actions
ACCEPTANCE_TEST: the single objective check that proves DONE (command / query / named
test)
DEFINITION_OF_DONE: binary checklist, each item independently verifiable

```

0.9 Glossary

- **Vertical:** a business type (e.g., `catering`, `plumber`). 30 active.
- **Preset:** `<slug>_front_desk` — the bundle of conversation blocks for a vertical.
- **Intake tree:** `<slug>_intake` — the slot-filling question tree for a vertical.
- **Tenant:** one business account in the multi-tenant system.
- **10DLC:** carrier registration required before A2P SMS can send in the US.
- **Runtime:** the compiled per-tenant checklist config derived from its preset.

1. MASTER STATUS TABLE (machine-readable — the single source of truth for “what can I claim now?”)

Legend:  DONE ·  IN_PROGRESS ·  BLOCKED ·  NOT_STARTED

ID	Title	Pass	Owner	Priority	Depends On	Status
T-000	Vertical intake trees (30 verticals)	1	Claude	HIGH	—	 DONE
T-001	Security credentials rotation	1	Human	CRITICAL	—	
T-002	10DLC re-registration + ENABLE_SMS flip	1	Mixed	CRITICAL	—	
T-003	Live voice validation call	1	Human	CRITICAL	—	
T-004	Stripe test-mode wiring	1	Human	CRITICAL	—	
T-005	Stripe live-mode + bank account	1	Human	CRITICAL	T-004	
T-006	Monitoring & alerting	1	Claude	HIGH	—	
T-007	Fix E2E test flakiness	1	Claude	HIGH	—	
T-008	Validate intake trees end-to-end	1	Mixed	HIGH	T-000	
T-009	Volume metering & tier caps	1	Claude	HIGH	T-004	
T-010	Schedule pattern ad-	1	Claude	MEDIUM	—	

ID	Title	Pass	Owner	Priority	Depends On	Status
	option verify					
T-011	Verify cost tracking ledger	1	Claude	MEDIUM	—	☐
T-012	Deployment checklist & automation	1	Claude	MEDIUM	—	☐
T-013	Full onboarding walk-through	1	Human	MEDIUM	T-003, T-008	☐
T-014	Dead code removal	1	Claude	LOW	T-001..T-013	☐
T-101	Onboarding wizard redesign	2	Claude	CRITICAL	T-013	☐
T-102	AI persona customization	2	Claude	HIGH	T-003	☐
T-103	Knowledge base builder UX	2	Claude	HIGH	—	☐
T-104	Dashboard UX polish	2	Claude	HIGH	T-013	☐
T-105	Vocabulary customization	2	Claude	HIGH	T-008	☐
T-106	Scheduler UX improvements	2	Claude	MEDIUM	—	☐
T-107		2	Claude	MEDIUM	T-005	☐

ID	Title	Pass	Owner	Priority	Depends On	Status
	Customer self-service booking page					
T-108	SMS & re-minder customization	2	Claude	MEDIUM	T-002	
T-109	Owner mobile PWA	2	Claude	MEDIUM	T-104	
T-110	In-app on-boarding checklist	2	Claude	MEDIUM	T-101	
T-201	Advanced analytics dashboard	3	Claude	HIGH	T-006	
T-202	Multi-location support	3	Claude	HIGH	T-009, T-005	
T-203	Team management (multi-user)	3	Claude	HIGH	—	
T-204	Zapier & outbound webhooks	3	Claude	MEDIUM	—	
T-205	Additional CRM integrations	3	Claude	MEDIUM	T-204	
T-206	Google Business Profile integration	3	Claude	MEDIUM	—	
T-207	Deposit payment collection	3	Claude	MEDIUM	T-005	

ID	Title	Pass	Owner	Priority	Depends On	Status
T-208	Demo / trial mode	3	Claude	HIGH	T-101	☐
T-301	White-label / re-seller program	4	Claude	HIGH	T-203, T-202	☐
T-302	Public API	4	Claude	MEDIUM	T-204	☐
T-303	Advanced AI voice features	4	Claude	MEDIUM	T-003	☐
T-304	Enterprise hardening (non-HIPAA)	4	Claude	MEDIUM	T-203, T-202	☐
T-305	Integration marketplace	4	Claude	LOW	T-302, T-204	☐

To claim a task: pick the highest-priority row whose `Depends On` are all ☒ and status is ☐. Set it , create branch `feat/T-XXX-<slug>` (or `fix/`), do the work, satisfy the Acceptance Test, open a PR, then set ☒ on merge.

PASS 1 — Foundation: Ready for First Paying Customer

Goal: System is operational, secure, billable, and validated on a real call.

Exit criteria (all must be provable):

- [] A new tenant can sign up, complete setup, and receive AI reception on a real call.
- [] Stripe charges a real card and the subscription gate flips (test-mode round-trip green).
- [] Monitoring fires an alert before a human notices an outage (proven by a triggered test alert).
- [] All 4 CI jobs green and stable across 3 consecutive runs.

T-000: Vertical intake trees (30 verticals)

STATUS: ☒ DONE (merged to main, PR #388)

OWNER: Claude-able

PRIORITY: HIGH

EFFORT: — (complete)

DEPENDS_ON: None

CONTEXT: Each of the 30 non-HIPAA verticals now has a dedicated slot-filling intake tree, block, and `_front_desk` preset. Kept here as the reference example of a completed work order.

FILES: `agent/src/checklist/verticalIntakeTrees.ts`, `trees.ts`, `blockLibrary.ts`, `presets.ts`, `shared/checklistPresetDerivation.ts`, `dashboard/lib/checklistPresets.ts`, `supabase/migrations/20260901000000_vertical_intake_preset_ids.sql`.

ACCEPTANCE_TEST: `npm test` && `cd agent` && `npm test` && `cd ../dashboard` && `npx vitest run` — all green; `presetCatalog.test.ts` asserts 33 presets; every platform tree is reachable by a preset.

DEFINITION_OF_DONE:

- [x] 30 `QuestionTreeDef` constants exist and are spread into `PLATFORM_TREE_LIBRARY`.
- [x] `defaultChecklistPresetIdForBusinessType('catering')` returns `catering_front_desk`.
- [x] All three suites green; merged to `main`.

T-001: Security credentials rotation

STATUS: ☐ NOT_STARTED

OWNER: Human-only (requires Railway + Supabase console access)

PRIORITY: CRITICAL

EFFORT: 1h

DEPENDS_ON: None

CONTEXT: A Railway team token (exposed 2026-06-12) and the Supabase DB password (exposed 2026-07-11 in a transcript) must be rotated. Until rotated, both are live compromise vectors.

FILES: none in-repo (console actions); update local `.env` and Railway env vars.

STEPS:

1. Railway → Team → Tokens → delete the 2026-06-12 token → create a new one.
2. Supabase → Database → reset password.
3. Update `DATABASE_URL` on Railway services `secretary-hq` and `secretary-hq-agent`, and in local `.env`.
4. Redeploy backend.

ACCEPTANCE_TEST (objective, runnable by anyone with prod URL):

```
curl -sf https://secretary-hq-production.up.railway.app/health | grep -q
'"status":"ok"' # exit 0
# AND: the old Railway token returns 401 when used: railway whoami --token <OLD> → fails
```

DEFINITION_OF_DONE:

- [] Old Railway token returns 401 (proven by a failed `railway whoami`).
- [] `/health` returns `{"status":"ok"}` after redeploy (new DB password works).
- [] No `password authentication failed` lines in backend logs for 10 min post-deploy.

T-002: 10DLC registration + ENABLE_SMS flip

STATUS: ☐ NOT_STARTED

OWNER: Mixed (Human does 10DLC registration at Telnyx; Claude does the gate + verification code)

PRIORITY: CRITICAL

EFFORT: Human 2h + carrier approval wait (days) · Claude 3h

DEPENDS_ON: None

CONTEXT: SMS is OFF **by design** — `ENABLE_SMS` defaults false in the agent/backend config schema

because US A2P SMS requires 10DLC brand+campaign registration. This is NOT a code bug. The task is: register 10DLC, then flip the gate, then prove one message actually delivers.

FILES: `agent/src/configSchema.ts` (or wherever `ENABLE_SMS` is defined — grep it), `src/services/telnyxSms.ts`, `src/services/smsService.ts`, `src/services/reminders/index.ts`, plus a new verification script `scripts/verify-sms-delivery.mjs`.

STEPS:

1. (Human) Register 10DLC brand + campaign in Telnyx portal; associate `+1 630-822-9086`; wait for approval.
2. (Human) Confirm `TELNYX_API_KEY` + `TELNYX_MESSAGING_PROFILE_ID` set on Railway.
3. (Claude) Add `scripts/verify-sms-delivery.mjs`: sends one SMS via the real `telnyxSms` path to a provided number, then polls `communications_history` for `status='delivered'`.
4. (Human) Set `ENABLE_SMS=true` on Railway once campaign approved.

ACCEPTANCE_TEST:

```
# After ENABLE_SMS=true and 10DLC approved:
node scripts/verify-sms-delivery.mjs --to <verified_phone>
# Script exits 0 ONLY when a communications_history row reaches
status='delivered' (not just 'sent').
```

DEFINITION_OF_DONE:

- [] 10DLC campaign shows “approved” in Telnyx (screenshot in PR).
- [] `scripts/verify-sms-delivery.mjs` exits 0 with a `delivered` receipt.
- [] A booked appointment produces a reminder row that reaches `status='delivered'` within one reminder tick.
- [] Unit test: SMS send path is a no-op when `ENABLE_SMS=false` (guard test in `smsService` tests).

T-003: Live voice validation call

STATUS: ☐ NOT_STARTED

OWNER: Human-only (requires two real phones)

PRIORITY: CRITICAL

EFFORT: 30m call + 1h analysis

DEPENDS_ON: None

CONTEXT: Production has **never booked an appointment on a real call** (5 calls, 0 bookings all-time). This validates the booking leg end-to-end. NOTE: there is no live human transfer on the tree path — escalation takes a message + urgent flag. Do NOT test “transfer to a person”; test “leave an urgent message.”

FILES: findings go to `docs/CALL_FIX_PLAN.md` (append a dated section).

STEPS:

1. Dashboard → Phone Assistant → set the escalation contact.
2. From a second phone, call `+1 630-822-9086`.
3. Book an appointment for a real time inside a shift window.
4. Trigger escalation (“this is urgent”) → verify a message row with urgency flag.

ACCEPTANCE_TEST (objective DB queries after the call):

```
-- 1 new voice session with a transcript:
SELECT count(*) FROM voice_sessions WHERE created_at > now() - interval '15 min';
-- >= 1
-- 1 appointment booked at the requested time for the test tenant:
SELECT count(*) FROM appointments WHERE created_at > now() - interval '15 min';
-- >= 1
-- escalation captured as an urgent message:
SELECT urgency FROM messages WHERE created_at > now() - interval '15 min';
-- 'urgent'
```

DEFINITION_OF_DONE:

- [] voice_sessions has the call with a non-empty transcript.
- [] appointments has the booking at the correct time for the correct tenant.
- [] Urgent message captured (no false “transferred” claim).
- [] Findings appended to docs/CALL_FIX_PLAN.md with the transcript.

T-004: Stripe test-mode wiring

STATUS: ☐ NOT_STARTED

OWNER: Human-only (Stripe dashboard + Railway env)

PRIORITY: CRITICAL

EFFORT: 2h

DEPENDS_ON: None

CONTEXT: Stripe checkout + webhook code exists (src/routes/billing.ts) but zero webhook endpoints are registered and price IDs are unset, so all /billing/* routes 503. No bank account needed for test mode.

FILES: none in-repo (config); optionally extend scripts/simulate.sh stripe .

STEPS:

1. Decide tier pricing (research suggests Solo \$99–129 / Growth \$199–249 / Pro \$349+).
2. Create 3 products + prices in Stripe **TEST** mode.
3. Register webhook https://secretary-hq-production.up.railway.app/billing/webhook (events: checkout.session.completed , invoice.payment_failed , customer.subscription.deleted); copy whsec_ .
4. Set Railway env: STRIPE_WEBHOOK_SECRET , STRIPE_SOLO_PRICE_ID , STRIPE_GROWTH_PRICE_ID , STRIPE_PRO_PRICE_ID .
5. Run a test checkout (card 4242 4242 4242 4242).

ACCEPTANCE_TEST:

```
./scripts/simulate.sh stripe      # path-check must pass
# Then a test-card checkout flips the tenant gate:
#   SELECT subscription_status FROM tenants WHERE id='<test_tenant>'; → 'active'
# Then cancel → webhook → SELECT ... → 'canceled'
```

DEFINITION_OF_DONE:

- [] /billing/* no longer 503 (returns a checkout URL for a valid price).
- [] Test checkout sets tenant subscription_status='active' .
- [] customer.subscription.deleted sets it back to canceled .
- [] Stripe dashboard shows webhook deliveries with 200 responses.

T-005: Stripe live-mode + bank account

STATUS: ☐ NOT_STARTED

OWNER: Human-only

PRIORITY: CRITICAL

EFFORT: 4h + bank setup

DEPENDS_ON: T-004

CONTEXT: Live payouts require an LLC bank account. Live-mode Stripe objects are separate from test-mode; new price IDs and a new `whsec_`.

STEPS:

1. Open Thinking Hammer LLC bank account; connect to Stripe.
2. Enable Stripe Tax (register IL nexus + customer states).
3. Recreate products/prices in LIVE mode; register LIVE webhook; copy new `whsec_`.
4. Swap all 5 Railway env vars to live values.

ACCEPTANCE_TEST:

```
# A real card (small amount, then refunded) completes checkout in LIVE mode:
# SELECT subscription_status FROM tenants WHERE id='<real_tenant>'; → 'active'
# Stripe LIVE dashboard shows the webhook delivered 200 for check-
out.session.completed.
```

DEFINITION_OF_DONE:

- [] Bank account connected (Stripe shows payouts enabled).
- [] Live checkout activates a real tenant.
- [] Live webhook deliveries return 200.
- [] `STRIPE_AUTO_TAX=true` and tax appears on the invoice.

T-006: Monitoring & alerting

STATUS: ☐ NOT_STARTED

OWNER: Claude-able (code) + Human (create alert channel)

PRIORITY: HIGH

EFFORT: 6–10h

DEPENDS_ON: None

CONTEXT: A reminder outage went undetected for 13 days while `/health` stayed green. We need metrics + alerts that fire on real failure conditions. Metrics already partly emit to a token-gated `/metrics`.

FILES: `src/routes/metrics.ts` (or wherever `/metrics` lives — `grep prom-client`), agent instrumentation in `agent/src/session/`, new `docs/ALERTS.md` rules, alert config (Better Stack recommended).

STEPS:

1. Confirm/instrument counters: `calls_total`, `call_outcome_total{outcome}`, `reminders_sent_total`, `reminders_skipped_total{reason}`, `errors_total{event}`, `turn_latency_ms`, `sms_sends_total{status}`, `webhook_signature_failures_total`.
2. Wire a scraper (Better Stack) to `/metrics`.
3. Encode alert rules (see below) as monitors.

ACCEPTANCE_TEST:

```
# Metrics endpoint exposes required series:
curl -s -H "Authorization: Bearer $METRICS_TOKEN" https://<backend>/metrics \
  | grep -E 'reminders_sent_total|turn_latency_ms|errors_total' | wc -l # >= 3
# Alert proof: deliberately increment errors_total{event="reminder_batch_failed"} 4x
# in staging → the configured monitor fires a notification (screenshot in PR).
```

DEFINITION_OF_DONE:

- [] All 8 metric series present in `/metrics` output.
- [] Alert rules committed to `docs/ALERTS.md`: `reminder_batch_failed>3/10min` (page), `sms failure>5%` (warn), `webhook_signature_failures>0/1h` (page), `turn_latency p95>3000ms` (warn).
- [] At least one alert proven to fire (notification screenshot in PR).

T-007: Fix E2E test flakiness

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: HIGH

EFFORT: 4–6h

DEPENDS_ON: None

CONTEXT: 3 tests intermittently redden `main` CI (which terminally SKIPS the Railway deploy). They assert wall-clock speed on a loaded runner rather than behavior. Known offenders:

`SetupWizard > shows success state with phone number after activation`;
`customer-preferences-config.spec.ts`; `purge-soft-deleted.test.ts` (partially fixed).

FILES: `dashboard/components/SetupWizard/SetupWizard.test.tsx`, `dashboard/e2e/customer-preferences-config.spec.ts`, `scripts/purge-soft-deleted.test.ts`.

STEPS:

1. Replace timeout-based waits with `waitFor()` on actual DOM/API state.
2. For the E2E spec, wait on the `update-config` network response, then assert value in Postgres before reload.
3. Remove any machine-speed assertions.

ACCEPTANCE_TEST:

```
# Each previously-flaky test passes 20 consecutive runs:
cd dashboard && npx vitest run SetupWizard --repeat 20 # 20/20 pass
npx playwright test customer-preferences-config --repeat-each 20 # 20/20 pass
```

DEFINITION_OF_DONE:

- [] Each named test passes 20/20 locally.
- [] 3 consecutive green CI runs on the PR.
- [] No `setTimeout` /hard-coded ms assertions remain in the touched tests (grep proves it).

T-008: Validate intake trees end-to-end

STATUS: ☐ NOT_STARTED

OWNER: Mixed (Claude runs simulator; Human confirms UI picker)

PRIORITY: HIGH

EFFORT: 2–3h

DEPENDS_ON: T-000

CONTEXT: The 30 trees are merged and unit-tested, but never exercised through the real onboarding + simulator path. Prove a sample of verticals resolve correctly and ask the right questions.

FILES: `agent/scripts/sim-questiontree.ts` (existing simulator), test tenants.

STEPS:

1. For each of 4 verticals (`catering` , `plumber` , `salon` , `real_estate`): create a tenant with that business_type.
2. Assert derivation + enabled blocks.
3. Run the simulator and confirm the intake questions fire.

ACCEPTANCE_TEST:

```
# Derivation + block wiring (unit-level, deterministic):
cd agent && npx vitest run checklistPresetDerivation
# For each sampled vertical, the enabled blocks include '<vertical>_intake':
#   node -e "import('../shared/checklistPresetDerivation.js').then(m=>{...assert...})"
# Simulator run reaches the intake tree (SIM_TRACE shows the intake node ids):
SIM_TRACE=1 npx tsx agent/scripts/sim-questiontree.ts --business catering | grep -q catering_
```

DEFINITION_OF_DONE:

- [] All 4 sampled verticals resolve to `<slug>_front_desk` .
- [] Enabled blocks for each include `<slug>_intake` .
- [] Simulator trace shows at least one intake node fired per vertical.
- [] No “tree not found” errors in the simulator log.

T-009: Volume metering & tier caps

STATUS: ☐ NOT_STARTED

OWNER: Claude-able (+ Human decides cap numbers)

PRIORITY: HIGH

EFFORT: 8-12h

DEPENDS_ON: T-004

CONTEXT: Tiers are flat subscriptions with no usage enforcement. Add per-tenant call counting + cap enforcement so an uncapped Solo tenant cannot run the platform into negative margin.

FILES: new migration `supabase/migrations/<ts>_tenant_usage_columns.sql` , `agent/src/index.ts` (session start), `src/routes/billing.ts` (webhook sets tier), dashboard usage view, tests.

STEPS:

1. Migration: add `subscription_tier` , `calls_this_month` , `month_reset_date` to `tenants` .
2. On session start, increment counter; reject over-cap calls with a spoken message.
3. Webhook maps `price_id` → tier; resets counter on new period.
4. Dashboard usage widget.

ACCEPTANCE_TEST:

```
npm test -- tenant-usage      # new suite
# Behavioral (real-DB test): seed Solo tenant at cap → next start_voice_session is rejected;
# upgrade tier → next session is accepted. Assertion in tests/services/usage-Caps.realdb.test.ts
```

DEFINITION_OF_DONE:

- [] Migration applies cleanly (`npm run db:migrate` + baseline updated + `verify:claude-md` green).

- [] Over-cap call rejected; logged `call_rejected_usage_limit_exceeded`.
- [] Webhook sets `subscription_tier` from `price_id` (test asserts).
- [] Dashboard shows calls used / cap / reset date.
- [] Real-DB test proves cap enforcement + upgrade path.

T-010: Schedule pattern adoption verify

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 2-4h

DEPENDS_ON: None

CONTEXT: Migration 20260820000000 added `employee_schedule_pattern` with a **deliberate no-back-fill** policy: existing tenants keep a clamped derived fallback until they next save hours. Do NOT invent patterns from historical rows ("row archaeology" was explicitly rejected). This task only verifies the adoption path works and documents the operational consequence.

FILES: `src/services/scheduling/extendSchedules.ts` (read-only understanding), `tests/services/extendSchedules.realdb.test.ts` (extend), `docs/RUNBOOK.md` (document the re-save trigger).

STEPS:

1. Write a real-DB test: tenant with no pattern + a far-future one-off shift → extend does NOT go weekend-only (clamp holds).
2. Then owner saves weekly hours → `employee_schedule_pattern` row appears → extend now projects the declared rule.
3. Document in RUNBOOK: "existing tenants adopt the rule on next hours save."

ACCEPTANCE_TEST:

```
npm test -- extendSchedules.realdb # includes the far-future-shift regression + adoption case
```

DEFINITION_OF_DONE:

- [] Real-DB test proves clamp prevents poisoning without a pattern (fails if clamp removed).
- [] Real-DB test proves saving hours writes the pattern and switches extend to declared-rule mode.
- [] RUNBOOK documents the re-save adoption trigger. No backfill script is added.

T-011: Verify cost tracking ledger

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 2-3h

DEPENDS_ON: None

CONTEXT: The AI cost ledger once undercounted 35× (tracked only 40-mini). It should now track all 4 legs. Verify and add a dashboard readout so pricing decisions use real numbers.

FILES: `src/services/aiCost/` (grep), a dashboard cost widget, tests.

ACCEPTANCE_TEST:

```
npm test -- aiCost
# Assertion: a simulated call records 4 cost components (voice LLM in/out, TTS chars,
STT seconds, summary LLM)
# and total is within [$0.03, $0.20] for a ~2-min call (sanity band, not a fixed
value).
```

DEFINITION_OF_DONE:

- [] Test proves all 4 cost legs are recorded per call.
- [] Per-call total falls in the sanity band for a synthetic 2-min call.
- [] Dashboard shows avg cost/call and a breakdown.

T-012: Deployment checklist & automation

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 3-4h

DEPENDS_ON: None

CONTEXT: Deploys have footguns (Railway "Wait for CI" is unversioned; migration ordering; SKIPPED-is-terminal). Capture a checklist + automate the mechanical checks.

FILES: new docs/DEPLOYMENT_CHECKLIST.md , new .github/workflows/pre-merge-checks.yml .

ACCEPTANCE_TEST:

```
# The workflow runs on PR and fails if CLAUDE.md drifts or a plaintext secret appears:
npm run verify:claude-md          # exits 0
git grep -nE 'sk_live_|whsec_[A-Za-z0-9]+' -- '!:docs/*' && exit 1 || exit 0 # no
secrets committed
```

DEFINITION_OF_DONE:

- [] docs/DEPLOYMENT_CHECKLIST.md covers pre-merge, migration order, post-deploy verification, and the 3 gotchas.
- [] .github/workflows/pre-merge-checks.yml runs drift + secret scan on every PR and is required.
- [] A deliberately-drifted CLAUDE.md makes the workflow fail (proof in PR).

T-013: Full onboarding walk-through

STATUS: ☐ NOT_STARTED

OWNER: Human-only (product acceptance)

PRIORITY: MEDIUM

EFFORT: 3-4h

DEPENDS_ON: T-003, T-008

CONTEXT: No one has completed the real onboarding as a customer. This produces the friction list that seeds PASS 2.

ACCEPTANCE_TEST (objective completion, not subjective quality):

A single tenant reaches "live" state with, verifiable in DB:

- ≥ 1 service, ≥ 1 employee, ≥ 1 resource, a weekly schedule, a persona, ≥ 1 KB doc, an active phone number.
- A real test call produced an appointment + transcript.
- Friction findings written to docs/UX_FRICTION_LOG.md (≥ 5 concrete, itemized issues).

DEFINITION_OF_DONE:

- [] Tenant reaches live state with all 7 setup artifacts present in DB.
- [] Test call booked an appointment.
- [] docs/UX_FRICTION_LOG.md created with ≥ 5 itemized findings, each tagged with the screen + step.

T-014: Dead code removal

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: LOW

EFFORT: 2-3h

DEPENDS_ON: T-001..T-013 (remove only after features stable)

CONTEXT: `ReminderProcessor` (unused parallel impl), commented debug code, unused route stubs.

ACCEPTANCE_TEST:

```
npm test && cd agent && npm test && cd ../dashboard && npx vitest run # all green
after removal
npx knip || true # (optional) report unused exports; removed items no longer listed
```

DEFINITION_OF_DONE:

- [] `ReminderProcessor` deleted; grep finds no references.
- [] All three suites still green.
- [] No behavior change (no migration, no route contract change).

PASS 2 — Product Depth: Self-Serve, Fast, and Sticky

Goal: A non-technical owner can onboard alone in under 10 minutes, customize the AI, and run their front desk from a phone — no support ticket required.

Exit criteria (all must be provable):

- [] A brand-new owner reaches "live" state (all 7 setup artifacts + 1 booked test call) in a single unbroken session, measured by a timestamped onboarding-events log ≤ 10 minutes end to end.
- [] Every customization surface (persona, vocabulary, KB, reminders) writes to the DB and is read back by the agent at call time (proven by a real/simulated call reflecting the change).
- [] Owner PWA passes Lighthouse PWA audit (installable) and renders the dashboard on a 390px viewport with no horizontal scroll.
- [] All 4 CI jobs green across 3 consecutive runs after each merge.

T-101: Onboarding wizard redesign

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: HIGH

EFFORT: 12-16h

DEPENDS_ON: T-013 (friction log seeds the redesign)

CONTEXT: Current SetupWizard loses users mid-flow (T-013 friction log documents where). Goal: a linear, resumable wizard with a persisted step pointer so an owner can close the tab and return to the same step. Each step must be independently completable and emit an onboarding event for timing.

FILES: dashboard/components/SetupWizard/, dashboard/lib/onboarding.ts (new step-state + event emitter), src/routes/onboarding.ts (persist onboarding_step + append onboarding_events), supabase/migrations/<new>_onboarding_events.sql (table onboarding_events(tenant_id, step, event, created_at) + tenants.onboarding_step).

STEPS:

1. Add migration for onboarding_events and tenants.onboarding_step.
2. Refactor wizard into a step registry (array of {id, isComplete(tenant), render}); derive current step from DB, not local state.
3. On entering/leaving each step, POST an event row.
4. Add a resume banner that deep-links to onboarding_step.

ACCEPTANCE_TEST:

```
cd dashboard && npx vitest run components/SetupWizard # wizard step-registry unit tests green
# Resumability (real-DB test in tests/routes/onboarding.test.ts):
#   Set tenants.onboarding_step='schedule' → GET /onboarding returns step='schedule' as current.
# Timing instrumentation present:
#   A completed onboarding writes >= 7 onboarding_events rows; MAX(created_at)-MIN(created_at) is computable.
npm test -- onboarding
```

DEFINITION_OF_DONE:

- [] Migration applied; onboarding_events + tenants.onboarding_step exist (verify via CLAUDE.md migration count gate).
- [] Closing and reopening the wizard resumes at the persisted step (test asserts DB-derived current step).
- [] Every step transition writes an event row; a full run yields ≥ 7 rows.
- [] All three suites green.

T-102: AI persona customization

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: HIGH

EFFORT: 8-10h

DEPENDS_ON: T-003 (persona plumbing stable)

CONTEXT: Owners must edit greeting, tone, business name pronunciation, and hold/handoff phrasing. The agent must read these at call time. No free-form LLM prompt injection — fields map to fixed slots in the greeting/closing nodes of the question tree.

FILES: `dashboard/components/PersonaEditor/` (new), `src/routes/persona.ts`, `agent/src/checklist/trees.ts` (greeting/closing node consumes persona fields), `supabase/migrations/`
`<new>_persona_fields.sql` (add columns to `personas`).

STEPS:

1. Migration: add `greeting_template`, `tone`, `business_name_spoken`, `escalation_phrase` to `personas`.
2. Build editor form with live text preview (no audio required).
3. Wire agent greeting/closing nodes to substitute persona fields.

ACCEPTANCE_TEST:

```
# Round-trip (real-DB test tests/routes/persona.test.ts):
# PUT /persona {greeting_template:'Thanks for calling {{business}}'} → 200
# GET /persona returns the saved template.
# Agent consumption (agent unit test agent/tests/persona.test.ts):
# Given a persona with business_name_spoken='Ace Plumbing', the rendered greeting
# node text contains 'Ace Plumbing'.
cd agent && npm test -- persona && cd ../dashboard && npx vitest run PersonaEditor
```

DEFINITION_OF_DONE:

- [] Persona fields persist and read back via API (test asserts round-trip).
- [] Agent greeting/closing text reflects persona fields (agent unit test asserts substituted string).
- [] No free-form prompt path introduced (grep: no persona field is concatenated into an LLM system prompt).

T-103: Knowledge base builder UX

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 10-12h

DEPENDS_ON: None

CONTEXT: Owners need to add FAQ/policy snippets the agent can answer from (hours, pricing ranges, service area, parking, etc.). Builder must support add/edit/delete and categorize entries. Agent answers “info” questions from these entries via exact-match/keyword lookup (no vector RAG required for MVP).

FILES: `dashboard/components/KnowledgeBase/` (new), `src/routes/knowledge.ts`,
`agent/src/checklist/infoLookup.ts` (new keyword lookup), `supabase/migrations/`
`<new>_kb_entries.sql` (`kb_entries(tenant_id, category, question, answer, keywords[])`).

STEPS:

1. Migration for `kb_entries`.
2. CRUD UI with category grouping and a required non-empty answer.
3. Agent info node: on an info question, match keywords → return the answer; else fall back to “I’ll take a message.”

ACCEPTANCE_TEST:

```
# CRUD (real-DB test tests/routes/knowledge.test.ts):
#   POST kb_entry → GET lists it → DELETE → GET no longer lists it.
# Lookup (agent unit test agent/tests/infoLookup.test.ts):
#   Given entry {keywords:['hours','open'], answer:'9 to 5'}, lookup('what are your
hours') returns '9 to 5';
#   lookup('do you sell cars') returns null (falls back to message).
cd agent && npm test -- infoLookup && cd ../dashboard && npx vitest run KnowledgeBase
```

DEFINITION_OF_DONE:

- [] KB CRUD persists (test asserts create/list/delete round-trip).
- [] Keyword lookup returns the right answer and null on miss (agent unit test).
- [] Miss path routes to message-taking (assertion in agent test).

T-104: Dashboard UX polish

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 8–10h

DEPENDS_ON: T-013

CONTEXT: The home dashboard must show, at a glance: calls today, appointments booked, missed/escalated, and upcoming reminders — each linking to detail. Replace any placeholder/mockered widgets with live queries.

FILES: `dashboard/components/Dashboard/`, `dashboard/lib/metrics.ts`, `src/routes/metrics.ts`.

STEPS:

1. Define a `/metrics/summary` endpoint returning the 4 counters for a date range.
2. Replace mocked widgets with data from the endpoint.
3. Add empty-state and loading states.

ACCEPTANCE_TEST:

```
# Endpoint (real-DB test tests/routes/metrics.test.ts):
#   Seed 3 calls, 2 appointments, 1 escalation today → GET /metrics/summary returns
{calls:3, appointments:2, escalations:1}.
# No mocks remain (grep gate):
grep -rn "mockData\|TODO: wire\|placeholder" dashboard/components/Dashboard && exit 1
|| echo "clean"
cd dashboard && npx vitest run Dashboard
```

DEFINITION_OF_DONE:

- [] `/metrics/summary` returns correct counts against seeded data (test asserts exact numbers).
- [] No `mockData` /placeholder strings remain in the Dashboard folder (grep gate passes).
- [] Each widget links to its detail view (component test asserts href).

T-105: Vocabulary customization

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 6–8h

DEPENDS_ON: T-008 (per-vertical intake trees stable)

CONTEXT: Owners must rename services/resources to their own words (e.g. “bay” vs “chair”, “consult” vs “appointment”) so the agent speaks their language. This maps display labels to the fixed tree slots; slot ids do not change.

FILES: `dashboard/components/Vocabulary/` (new), `src/routes/vocabulary.ts`, `agent/src/checklist/labels.ts` (new label resolver), `supabase/migrations/<new>_vocabulary_overrides.sql` (`vocab_overrides(tenant_id, slot_key, label)`).

STEPS:

1. Migration for `vocab_overrides`.
2. UI listing overridable slot keys with a label input.
3. Agent resolves prompt labels through overrides at render time.

ACCEPTANCE_TEST:

```
# Round-trip (real-DB test tests/routes/vocabulary.test.ts):
# PUT override {slot_key:'resource', label:'bay'} → GET returns 'bay'.
# Agent render (agent unit test agent/tests/labels.test.ts):
# With override resource->'bay', the resource-selection node prompt contains 'bay',
# not 'resource'.
cd agent && npm test -- labels && cd ../dashboard && npx vitest run Vocabulary
```

DEFINITION_OF_DONE:

- [] Overrides persist and read back (test asserts round-trip).
- [] Agent prompt text uses the override label (agent unit test asserts substituted word).
- [] Slot ids/tree structure unchanged (grep: no slot_key renamed in trees).

T-106: Scheduler UX

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 10-12h

DEPENDS_ON: None

CONTEXT: Owners need a visual weekly calendar to view/edit appointments and set business hours, breaks, and blackout dates. Availability edits must be respected by the booking logic the agent uses.

FILES: `dashboard/components/Scheduler/` (new), `src/routes/schedule.ts`, `src/services/availability.ts`, `supabase/migrations/<new>_blackout_dates.sql` (`blackout_dates(tenant_id, date, reason)`).

STEPS:

1. Migration for `blackout_dates`.
2. Weekly grid UI; edit hours/breaks; add blackout dates.
3. `availability.ts` excludes blackout dates + breaks from bookable slots.

ACCEPTANCE_TEST:

```
# Availability (real-DB test tests/services/availability.test.ts):
# Add blackout_date=2026-12-25 → getSlots('2026-12-25') returns [] (empty);
# getSlots on an open day returns >0 slots and excludes break windows.
cd dashboard && npx vitest run Scheduler
npm test -- availability
```

DEFINITION_OF_DONE:

- [] Blackout dates persist (migration gate + round-trip test).
- [] `getSlots` returns [] on a blackout date and excludes breaks (assertions with exact expected arrays).
- [] Scheduler component renders seeded appointments (component test).

T-107: Customer self-service booking pageSTATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 12-14h

DEPENDS_ON: T-005 (calendar/booking core stable)

CONTEXT: A public per-tenant page (`/book/:tenantSlug`) where a caller's customer can self-book using the same availability engine the agent uses. Must prevent double-booking against agent-created appointments.

FILES: `dashboard/app/book/[slug]/` (new public route), `src/routes/publicBooking.ts` , reuse `src/services/availability.ts` , `supabase/migrations/<new>_tenant_slug.sql` (`tenants.slug` unique).

STEPS:

1. Migration: add unique `tenants.slug` .
2. Public availability + create-appointment endpoints (rate-limited, no auth, tenant-scoped by slug).
3. Booking UI: pick service → slot → contact info → confirm.

ACCEPTANCE_TEST:

```
# Double-book guard (real-DB test tests/routes/publicBooking.test.ts):
#   Book slot S via public API → second booking of S returns 409.
#   A slot already taken by an agent-created appointment is NOT offered by GET public
#   availability.
# Tenant scoping:
#   GET /book/<slugA> availability never returns tenantB's slots.
cd dashboard && npx vitest run book
npm test -- publicBooking
```

DEFINITION_OF_DONE:

- [] `tenants.slug` unique migration applied.
- [] Public booking creates an appointment and blocks the slot (409 on re-book — test asserts status).
- [] Availability excludes agent-booked slots (test asserts slot absent).
- [] Endpoints are rate-limited (test asserts 429 after N requests).

T-108: SMS & reminder customizationSTATUS: ☐ NOT_STARTED

OWNER: Mixed (code = Claude; 10DLC delivery = human)

PRIORITY: MEDIUM

EFFORT: 8-10h

DEPENDS_ON: T-002 (SMS/10DLC enablement)

CONTEXT: Owners edit reminder timing (e.g. 24h + 2h before) and message templates. Templates render with appointment variables. NOTE: actual SMS send stays gated on `ENABLE_SMS` (off until

10DLC approved — not a bug). This task delivers the config + template rendering; live delivery verification is the human half under T-002.

FILES: `dashboard/components/Reminders/` (new), `src/routes/reminders.ts`, `src/services/reminders/index.ts` (consume templates + timing), `supabase/migrations/<new>_reminder_config.sql` (`reminder_config(tenant_id, offsets_minutes[], sms_template, voice_template)`).

STEPS:

1. Migration for `reminder_config`.
2. UI to set offsets + templates with variable insertion (`{{name}}`, `{{time}}`, `{{service}}`).
3. Reminder service renders template + schedules per configured offsets.

ACCEPTANCE_TEST:

```
# Template render (unit test tests/services/reminders.test.ts):
#   renderTemplate('Hi {{name}}, {{time}}', {name:'Sam', time:'3pm'}) === 'Hi Sam, 3pm'.
# Scheduling (real-DB test):
#   offsets=[1440,120] on an appt at T → two reminder rows scheduled at T-24h and T-2h.
# Gate respected:
#   With ENABLE_SMS=false, scheduled SMS rows have status='suppressed' (not 'sent').
npm test -- reminders && cd dashboard && npx vitest run Reminders
```

DEFINITION_OF_DONE:

- [] `reminder_config` persists offsets + templates (round-trip test).
- [] Template renderer substitutes all variables (unit test with exact expected string).
- [] Configured offsets produce the exact scheduled reminder rows (test asserts count + times).
- [] `ENABLE_SMS=false` suppresses send (status='suppressed'); does not throw.

T-109: Owner mobile PWA

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 10-12h

DEPENDS_ON: T-104 (dashboard widgets live)

CONTEXT: The dashboard must be installable and usable on a phone: manifest, service worker, responsive layout, no horizontal scroll at 390px. Owners check calls/appointments on the go.

FILES: `dashboard/public/manifest.webmanifest`, `dashboard/app/sw.ts` (or next-pwa config), responsive CSS across `dashboard/components/`.

STEPS:

1. Add web app manifest (name, icons, display=standalone, theme).
2. Register a service worker (offline shell for the dashboard route).
3. Fix responsive breakpoints; verify no overflow at 390px.

ACCEPTANCE_TEST:

```
# Build + Lighthouse PWA (headless):
cd dashboard && npm run build && npx serve -s out -l 4321 &
npx lighthouse http://localhost:4321 --only-categories=pwa --output=json --output-
path=/tmp/lh.json --chrome-flags="--headless"
node -e "const r=require('/tmp/lh.json'); if(r.categories.pwa.score<0.9) pro-
cess.exit(1)" # PWA score >= 0.9
# Responsive (playwright test dashboard/tests/responsive.spec.ts):
# At viewport 390x844, document.scrollingElement.scrollWidth <= 390 on the dashboard
route.
```

DEFINITION_OF_DONE:

- [] Manifest + service worker present; app is installable (Lighthouse PWA score ≥ 0.9 — script exits 0).
- [] No horizontal scroll at 390px (playwright asserts scrollWidth $\leq$ viewport).
- [] Dashboard functions offline-shell (SW registered; verified in test).

T-110: In-app onboarding checklist

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: LOW

EFFORT: 6–8h

DEPENDS_ON: T-101 (wizard step model)

CONTEXT: A persistent checklist widget showing setup progress (services, employees, schedule, persona, KB, phone number, first call) with live completion state derived from the DB — not a static list. Drives the “live in under 10 minutes” promise.

FILES: `dashboard/components/OnboardingChecklist/` (new), reuse `dashboard/lib/onboarding.ts` step registry from T-101.

STEPS:

1. Compute each item’s completion from the same `isComplete(tenant)` predicates used by the wizard.
2. Render checklist with checkmarks + deep links to the incomplete step.
3. Hide the widget once all items complete.

ACCEPTANCE_TEST:

```
# Derivation (unit test dashboard/tests/onboardingChecklist.test.ts):
# Given a tenant with services+employees only, checklist shows exactly those 2
# complete, 5 incomplete.
# Given all 7 artifacts present, isAllComplete(tenant) === true and the widget
# renders null.
cd dashboard && npx vitest run onboardingChecklist
```

DEFINITION_OF_DONE:

- [] Completion state is DB-derived via shared predicates (no hardcoded booleans — grep gate).
- [] Checklist reflects exact complete/incomplete counts (unit test with fixtures).
- [] Widget hides at 100% (test asserts null render).

PASS 3 — Scale & Integrations: More Value Per Tenant

Goal: Grow revenue per tenant with analytics, multi-location, teams, and outbound integrations — without adding any HIPAA/PHI vertical.

Exit criteria (all must be provable):

- [] A tenant with 2+ locations and 2+ staff logins operates each location independently (data isolation proven by test).
- [] At least one outbound integration (Zapier/webhook) delivers an event to an external endpoint (proven by a received test payload).
- [] Analytics endpoints return correct aggregates against seeded data (exact-number assertions).
- [] All 4 CI jobs green across 3 consecutive runs after each merge.

T-201: Advanced analytics

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 12-14h

DEPENDS_ON: T-006 (metrics/observability base)

CONTEXT: Owners need trend reporting: calls over time, booking conversion rate, peak call hours, no-show rate, revenue booked (if price known). All from existing tables; no new external service.

FILES: `dashboard/components/Analytics/` (new), `src/routes/analytics.ts`, `src/services/analytics.ts`, `supabase/migrations/<new>_analytics_indexes.sql` (indexes on `calls.created_at`, `appointments.status`).

STEPS:

1. Add indexes to support time-range aggregation.
2. Build `analytics.ts` aggregators (calls/day, conversion = appointments/calls, no-show rate, hour histogram).
3. Charts UI with a date-range picker.

ACCEPTANCE_TEST:

```
# Aggregators (real-DB test tests/services/analytics.test.ts):
# Seed 10 calls, 4 appointments, 1 no-show → conversionRate()===0.4,
# noShowRate()===0.25.
# callsByDay() over a 3-day seed returns an array of length 3 with exact counts.
npm test -- analytics && cd dashboard && npx vitest run Analytics
```

DEFINITION_OF_DONE:

- [] Aggregators return exact values against seeded data (tests assert numbers, not ranges).
- [] Indexes present (migration gate).
- [] Charts render for a selected range (component test with fixture).

T-202: Multi-location support

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 16-20h

DEPENDS_ON: T-009, T-005

CONTEXT: One tenant, many locations — each with its own schedule, resources, phone number, and appointments. Data must be isolated per location; cross-location reads must be explicit. This is a schema + scoping change touching booking and availability.

FILES: `supabase/migrations/<new>_locations.sql` (`locations(tenant_id, name, ...)` , add `location_id` FK to `resources` , `employees` , `appointments` , `phone_numbers`), `src/services/availability.ts` , `src/routes/*` (location scoping), `dashboard/components/LocationSwitcher/` (new).

STEPS:

1. Migration: `locations` table + `location_id` FKs (backfill existing rows to a default location).
2. Thread `location_id` through availability + booking queries.
3. Location switcher in the dashboard; all lists filter by active location.

ACCEPTANCE_TEST:

```
# Isolation (real-DB test tests/services/multiLocation.test.ts):
#   Seed locationA + locationB with distinct appointments.
#   getAppointments(locationA) returns only A's rows; count matches; no B rows
#   present.
#   getSlots(locationA) reflects only A's schedule/blackouts.
# Backfill safety:
#   After migration, every pre-existing appointment has a non-null location_id.
npm test -- multiLocation
```

DEFINITION_OF_DONE:

- [] `locations` + `location_id` FKs applied; existing rows backfilled (test asserts no null `location_id`).
- [] Availability + booking are location-scoped (test asserts A-only results).
- [] Dashboard filters by active location (component test).

T-203: Team management (staff logins & roles)

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 12-14h

DEPENDS_ON: None

CONTEXT: A tenant needs multiple user logins with roles (owner, manager, staff) and permission gating (e.g. only owner edits billing). Invitations by email; role checked server-side on every protected route.

FILES: `supabase/migrations/<new>_users_roles.sql` (`tenant_users(tenant_id, user_id, role)` , `invitations(tenant_id, email, role, token, expires_at)`), `src/middleware/authorize.ts` (new role guard), `src/routes/team.ts` , `dashboard/components/Team/` (new).

STEPS:

1. Migration for `tenant_users` + `invitations` .
2. `authorize(role)` middleware; apply to protected routes.
3. Invite flow (create invite → accept via token → row in `tenant_users`).

ACCEPTANCE_TEST:

```
# Authorization (real-DB test tests/middleware/authorize.test.ts):
#   A 'staff' user calling PUT /billing → 403; an 'owner' → 200.
# Invite flow (real-DB test tests/routes/team.test.ts):
#   POST /team/invite → GET accept with token → tenant_users has the new (user, role)
#   row;
#   expired token → 410.
npm test -- authorize team
```

DEFINITION_OF_DONE:

- [] tenant_users + invitations applied (migration gate).
- [] Role guard blocks under-privileged calls (403) and allows privileged (test asserts both).
- [] Invite accept creates the membership row; expired token rejected (test asserts 410).

T-204: Zapier / outbound webhooks

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 10-12h

DEPENDS_ON: None

CONTEXT: Emit tenant-configurable webhooks on key events (appointment.created, call.completed, escalation.raised) so owners can pipe into Zapier/Make/CRM. Must sign payloads (HMAC) and retry on failure with backoff.

FILES: supabase/migrations/<new>_webhooks.sql (webhook_endpoints(tenant_id, url, secret, events[]) , webhook_deliveries(...)), src/services/webhooks.ts (new dispatcher), event emit points in booking/call/escalation services.

STEPS:

1. Migration for endpoints + deliveries.
2. Dispatcher: sign with HMAC-SHA256, POST, record delivery, retry with backoff on non-2xx.
3. Emit events at the three source points.

ACCEPTANCE_TEST:

```
# Delivery + signature (test tests/services/webhooks.test.ts with a local receiver):
#   Configure endpoint → create an appointment → receiver gets a POST whose
#   X-Signature header verifies against the secret and payload.
# Retry:
#   Receiver returns 500 twice then 200 → delivery row shows attempts=3, final
#   status='delivered'.
npm test -- webhooks
```

DEFINITION_OF_DONE:

- [] Endpoints/deliveries tables applied (migration gate).
- [] A configured event produces a signed, verifiable POST (test verifies HMAC).
- [] Failed deliveries retry with backoff and record attempts (test asserts attempts=3).

T-205: CRM integrations

STATUS: ☐ NOT_STARTED

OWNER: Mixed (code = Claude; CRM app credentials/approval = human)

PRIORITY: LOW

EFFORT: 14-16h

DEPENDS_ON: T-204 (webhook/event backbone)

CONTEXT: Push contacts/appointments to one CRM (start with HubSpot or Jobber). Uses the T-204 event backbone + a CRM adapter. Human half: registering the OAuth app + obtaining credentials.

FILES: `src/integrations/crm/` (new adapter interface + one concrete adapter), `src/routes/integrations.ts`, `supabase/migrations/<new>_crm_connections.sql` (`crm_connections(tenant_id, provider, access_token, refresh_token)`).

STEPS:

1. Define `CrmAdapter` interface (`upsertContact`, `createAppointment`).
2. Implement one adapter against the provider sandbox.
3. Subscribe to T-204 events → call adapter.

ACCEPTANCE_TEST:

```
# Adapter contract (unit test with mocked HTTP tests/integrations/crm.test.ts):
#   upsertContact(payload) issues the exact provider request (method+path+body
#   asserted against fixture).
#   A 401 triggers refresh-token flow then retry (asserted).
# Integration (sandbox, human-run once):
#   A real appointment.created event creates a matching record in the CRM sandbox
#   (screenshot in PR).
npm test -- crm
```

DEFINITION_OF_DONE:

- [] `CrmAdapter` interface + one adapter implemented; unit tests assert exact request shapes.
- [] Token refresh path covered by test.
- [] One end-to-end sandbox record created (human verification captured in PR).

T-206: Google Business Profile integration

STATUS: ☐ NOT_STARTED

OWNER: Mixed (code = Claude; Google app verification = human)

PRIORITY: LOW

EFFORT: 10-12h

DEPENDS_ON: None

CONTEXT: Sync business hours and let the agent reference GBP info; optionally post booking links.

Human half: Google Cloud project + OAuth consent verification.

FILES: `src/integrations/gbp/` (new), `src/routes/integrations.ts`, `supabase/migrations/<new>_gbp_connections.sql`.

STEPS:

1. OAuth connect + store tokens.
2. Pull hours → reconcile with tenant schedule (report mismatches; do not auto-overwrite).
3. Optional: post booking URL to the profile.

ACCEPTANCE_TEST:

```
# Reconciliation (unit test tests/integrations/gbp.test.ts with mocked API):
#   GBP hours {Mon 9-5} vs tenant {Mon 9-6} → reconcile() returns exactly one mismatch
#   entry {day:'Mon'}.
# Token storage round-trip test.
npm test -- gbp
```

DEFINITION_OF_DONE:

- [] OAuth connect stores/reads tokens (round-trip test).
- [] `reconcile()` reports exact mismatches without overwriting (test asserts mismatch list).
- [] Live connect verified once by human (captured in PR).

T-207: Deposit / prepay at bookingSTATUS: ☐ NOT_STARTED

OWNER: Mixed (code = Claude; live Stripe = human)

PRIORITY: MEDIUM

EFFORT: 12-14h

DEPENDS_ON: T-005 (booking core)

CONTEXT: Optionally require a deposit to confirm a booking (reduces no-shows). Uses Stripe Payment Intents; booking is held as 'pending' until payment succeeds, then 'confirmed'. Refund on cancel within policy window.

FILES: `src/services/deposits.ts` (new), `src/routes/publicBooking.ts` (deposit branch), `supabase/migrations/<new>_deposits.sql` (`deposits(appointment_id, amount, status, payment_intent_id)`).

STEPS:

1. Migration for `deposits`.
2. On deposit-required service, create `PaymentIntent`; hold appointment 'pending'.
3. Webhook `payment_intent.succeeded` → flip to 'confirmed'; cancel path → refund within window.

ACCEPTANCE_TEST:

```
# State machine (real-DB test tests/services/deposits.test.ts, Stripe test mode):
# Create deposit-required booking → appointment.status='pending', depos-
# it.status='requires_payment'.
# Simulate payment_intent.succeeded webhook → appointment='confirmed', depos-
# it='paid'.
# Cancel within window → refund issued, deposit='refunded'.
npm test -- deposits
```

DEFINITION_OF_DONE:

- [] `deposits` table applied (migration gate).
- [] Pending→confirmed transition driven by webhook (test asserts states).
- [] Refund-on-cancel within window (test asserts 'refunded').
- [] Live-mode smoke: one real deposit + refund (human, captured in PR).

T-208: Demo / trial modeSTATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: MEDIUM

EFFORT: 8-10h

DEPENDS_ON: T-101 (onboarding), T-000 (verticals)

CONTEXT: A prospect can spin up a fully-seeded demo tenant for a chosen vertical (sample services, schedule, KB, persona) and place a simulated call — no billing, auto-expires in 7 days. Drives conversion.

FILES: `src/services/demoTenant.ts` (new seeder), `src/routes/demo.ts`, `supabase/migrations/<new>_demo_flag.sql` (`tenants.is_demo`, `tenants.expires_at`), reuse simulator for the fake call.

STEPS:

1. Migration: `is_demo` , `expires_at` .
2. Seeder builds a complete demo tenant for a given vertical.
3. A scheduled cleanup deletes expired demo tenants.

ACCEPTANCE_TEST:

```
# Seeder completeness (real-DB test tests/services/demoTenant.test.ts):
#   createDemo('plumbing') yields a tenant with >=1 service, employee, resource,
#   schedule, persona, KB entry, phone (all 7 present).
#   is_demo=true and expires_at = created+7d.
# Cleanup:
#   A demo tenant with expires_at in the past is removed by cleanupExpiredDemos() (row
#   count drops to 0).
npm test -- demoTenant
```

DEFINITION_OF_DONE:

- [] Demo flag/expiry migration applied.
- [] Seeder produces all 7 setup artifacts (test asserts each present).
- [] Expired demos are purged by cleanup (test asserts deletion).
- [] Demo tenants never hit billing (grep/test: billing routes reject is_demo).

PASS 4 — Platform: White-Label, API, and Enterprise (Non-HIPAA)

Goal: Turn the product into a platform others build on — white-label resale, a public API, richer voice, and enterprise-grade operations. HIPAA/PHI is explicitly OUT of scope (no medical verticals, no PHI handling tier).

Exit criteria (all must be provable):

- [] A reseller can brand and operate the product for their own clients (branding + tenant isolation proven by test).
- [] The public API is documented, authenticated, versioned, and covered by contract tests.
- [] Enterprise controls (audit log, SSO, backups/restore) verified by test — WITHOUT introducing any HIPAA/PHI feature.
- [] All 4 CI jobs green across 3 consecutive runs after each merge.

T-301: White-label / reseller mode

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: LOW

EFFORT: 20-24h

DEPENDS_ON: T-203 (roles), T-202 (multi-location)

CONTEXT: A reseller (“partner”) owns many client tenants under its own brand (logo, colors, custom domain, sender identity). Partner-level admin manages client tenants; client data stays isolated per tenant.

FILES: `supabase/migrations/<new>_partners.sql` (`partners(...)` , `tenants.partner_id` , `branding(partner_id, logo_url, primary_color, domain)`), `src/middleware/branding.ts` (resolve brand by host), `dashboard/` theming from branding, `src/routes/partner.ts` .

STEPS:

1. Migration for `partners` + `branding` + `tenants.partner_id`.
2. Resolve brand by request host; theme dashboard from branding.
3. Partner admin: list/manage its client tenants only.

ACCEPTANCE_TEST:

```
# Brand resolution (test tests/middleware/branding.test.ts):
#   Request host 'acme.example.com' → resolves partner 'acme' branding (logo/color as-
#   asserted).
# Isolation (real-DB test tests/routes/partner.test.ts):
#   Partner A admin lists tenants → only partner_id=A tenants returned; never partner
#   B's.
npm test -- branding partner
```

DEFINITION_OF_DONE:

- [] `partners` / `branding` / `partner_id` applied (migration gate).
- [] Brand resolves by host (test asserts branding fields).
- [] Partner admin sees only its own tenants (test asserts isolation).

T-302: Public APISTATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: LOW

EFFORT: 16–20h

DEPENDS_ON: T-204 (event backbone)

CONTEXT: A documented, versioned REST API (`/api/v1/*`) with API-key auth and per-key rate limits, exposing appointments, calls, and availability. OpenAPI spec + contract tests are part of the deliverable.

FILES: `src/api/v1/` (new), `src/middleware/apiKey.ts`, `supabase/migrations/<new>_api_keys.sql` (`api_keys(tenant_id, key_hash, scopes[], rate_limit)`), `docs/openapi.yaml` (new).

STEPS:

1. Migration for `api_keys` (store hash, never plaintext).
2. API-key middleware (hash lookup + scope + rate limit).
3. `/api/v1` endpoints + `openapi.yaml`; contract tests validate responses against the spec.

ACCEPTANCE_TEST:

```
# Auth + scope (real-DB test tests/api/apiKey.test.ts):
#   No key → 401; valid key without scope → 403; valid scoped key → 200.
#   Exceeding rate_limit → 429.
# Contract (test tests/api/contract.test.ts):
#   Each /api/v1 response validates against docs/openapi.yaml (schema validation
#   passes).
npx @redocly/cli lint docs/openapi.yaml # spec is valid (exit 0)
npm test -- api
```

DEFINITION_OF_DONE:

- [] `api_keys` applied; keys stored hashed (test: no plaintext key in DB).
- [] Auth/scope/rate-limit enforced (tests assert 401/403/429/200).
- [] `openapi.yaml` lints clean and responses validate against it (contract tests pass).

T-303: Advanced voice (barge-in, warm phrasing, latency)

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: LOW

EFFORT: 16–20h

DEPENDS_ON: T-003 (voice/persona base)

CONTEXT: Improve naturalness within the question-tree model (NOT free-form LLM): support barge-in (caller interrupts TTS), filler/acknowledgement phrasing between slots, and measurably lower turn latency. Measured via the simulator's trace timings.

FILES: `agent/src/voice/` (barge-in handling), `agent/src/checklist/trees.ts` (acknowledgement phrasing slots), `agent/src/sim/` (latency trace assertions).

STEPS:

1. Barge-in: stop TTS on detected speech; resume tree at the pending node.
2. Add acknowledgement phrasing between slot transitions (still tree-driven).
3. Instrument per-turn latency in the simulator trace.

ACCEPTANCE_TEST:

```
# Barge-in (agent test agent/tests/bargeIn.test.ts):
#   Simulated speech during TTS sets state to 'interrupted' and re-enters the same
#   pending node (no slot lost).
# Latency (agent test agent/tests/latency.test.ts):
#   Simulated turn's decision latency (excluding network) < 300ms (asserted against
#   trace).
# Non-regression: no free-form LLM path added (grep: tree remains the control flow).
cd agent && npm test -- bargeIn latency
```

DEFINITION_OF_DONE:

- [] Barge-in interrupts TTS and preserves the pending slot (test asserts no slot loss).
- [] Acknowledgement phrasing present and tree-driven (test asserts phrasing without new LLM call).
- [] Per-turn decision latency under threshold (test asserts < 300ms from trace).

T-304: Enterprise hardening (non-HIPAA)

STATUS: ☐ NOT_STARTED

OWNER: Mixed (code = Claude; SSO IdP setup = human)

PRIORITY: LOW

EFFORT: 20–24h

DEPENDS_ON: T-203 (roles), T-202 (multi-location)

CONTEXT: Enterprise operational controls WITHOUT any HIPAA/PHI scope: immutable audit log of sensitive actions, SSO (SAML/OIDC) login, and tested backup/restore. NO medical vertical, NO PHI data class, NO BAA workflow — those remain permanently out of scope for this product.

FILES: `supabase/migrations/<new>_audit_log.sql` (`audit_log(tenant_id, actor, action, target, created_at)` append-only), `src/middleware/audit.ts`, `src/auth/sso/` (new), `scripts/backup.sh` + `scripts/restore.sh`.

STEPS:

1. Append-only audit log; write on billing/role/webhook/config changes.
2. SSO (OIDC first) login path mapping to a `tenant_user` role.

3. Backup + restore scripts; a restore test proves data returns intact.

ACCEPTANCE_TEST:

```
# Audit (real-DB test tests/middleware/audit.test.ts):
#   A role change writes exactly one audit_log row {action:'role.updated', actor, target};
#   audit rows cannot be updated/deleted (attempt raises/denied).
# SSO (test tests/auth/sso.test.ts, mocked IdP):
#   A valid OIDC assertion logs in and maps to the expected tenant_user role.
# Backup/restore (integration script):
#   backup.sh → drop a row → restore.sh → the row is present again (asserted).
# Scope guard:
grep -rin "hipaa\\|\\bphi\\b\\|protected health" src agent | grep -v "no HIPAA\\|out of scope" && exit 1 || echo "no HIPAA scope present"
npm test -- audit sso
```

DEFINITION_OF_DONE:

- [] Append-only audit log written on sensitive actions; mutation denied (tests assert both).
- [] SSO/OIDC login maps to a role (test asserts mapping).
- [] Backup/restore round-trip restores data (integration test asserts row returns).
- [] No HIPAA/PHI feature introduced (grep scope guard exits 0).

T-305: Integration marketplace

STATUS: ☐ NOT_STARTED

OWNER: Claude-able

PRIORITY: LOW

EFFORT: 16-20h

DEPENDS_ON: T-302 (public API), T-204 (webhooks)

CONTEXT: A catalog where tenants browse and enable integrations (built on the T-204 webhook + T-302 API primitives). Each integration is a manifest (name, events, config schema); enabling one provisions the underlying webhook/connection.

FILES: `src/marketplace/` (manifest loader + registry), `src/routes/marketplace.ts`, `dashboard/components/Marketplace/` (new), `supabase/migrations/<new>_tenant_integrations.sql` (`tenant_integrations(tenant_id, integration_key, config, enabled)`).

STEPS:

1. Define an integration manifest schema + registry loader.
2. Enabling an integration validates config against its schema and provisions the webhook/connection.
3. Marketplace UI lists manifests; toggle enable/disable.

ACCEPTANCE_TEST:

```
# Registry + enable (real-DB test tests/marketplace/enable.test.ts):
#   Enabling 'zapier' with valid config creates a tenant_integrations row (enabled=true)
#   and provisions a webhook_endpoint; invalid config → 422 and no row created.
#   Disabling removes/deactivates the underlying webhook.
npm test -- marketplace && cd dashboard && npx vitest run Marketplace
```

DEFINITION_OF_DONE:

- [] Manifest schema + registry loader implemented (unit test loads + validates a manifest).

- [] Enable provisions the backing webhook/connection; invalid config rejected 422 (tests assert both).
- [] Disable deactivates the backing resource (test asserts state).

CLOSING — Backlog Summary & How to Claim a Task

Task counts by pass

Pass	Range	Count	Done	Remaining
PASS 1 — Foundation	T-000 ... T-014	15	1 (T-000)	14
PASS 2 — Product Depth	T-101 ... T-110	10	0	10
PASS 3 — Scale & Integrations	T-201 ... T-208	8	0	8
PASS 4 — Platform	T-301 ... T-305	5	0	5
Total		38	1	37

Owner split (who does what)

- **Claude-able** (pure code, no external account needed): the implementer can start and finish these autonomously.
- **Human-only** (needs an external account/phone/bank/live call): T-001 (rotate creds), T-002 (10DLC/SMS live), the live-mode half of T-004 (real Stripe card), T-012 (real test call).
- **Mixed** (Claude writes code; human does the external step): T-108, T-205, T-206, T-207, T-304.

How to claim a task (implementer workflow)

1. **Pick** the lowest-numbered task whose `DEPENDS_ON` are all  DONE in the Master Status Table (Section 1).
2. **Read** its CONTEXT + Section 0 (Complete Context) so you understand the goal path and constraints (no HIPAA verticals; voice = question trees not free-form LLM; SMS off until 10DLC; no live human transfer on tree path).
3. **Branch** `feat/<task-id>-<slug>` off `main`. Never commit to `main` directly (branch protection requires a PR with 4 green CI jobs).
4. **Implement** the STEPS, touching the listed FILES.
5. **Prove it** by running the exact ACCEPTANCE_TEST commands. The task is done ONLY when every command exits 0 / returns the stated value — these are objective and non-negotiable.
6. **Check off** every box in DEFINITION_OF_DONE.
7. **Open a PR**; ensure all 4 CI jobs are green. Merge = deploy to Railway.
8. **Update** the task's STATUS to  DONE (with the PR #) in both the Master Status Table and the task header.

Completion status is machine-readable

- Section 1's STATUS column is the single source of truth. ☐ NOT_STARTED / ☒ IN_PROGRESS / ☒ DONE .
- A task may only flip to ☒ DONE when its ACCEPTANCE_TEST passes — no subjective judgment is permitted anywhere in this document.

Last updated: 2026-09-01.